

# 近端策略优化 (PPO)

## 在线策略梯度方法

深圳技术大学

2026 年 5 月 30 日

# 目录

- 1 策略梯度
- 2 动机
- 3 重要性采样
- 4 信赖域
- 5 PPO
- 6 直接偏好优化 (DPO)
- 7 应用
- 8 附录：信息论基础

# 目录

- 1 策略梯度
- 2 动机
- 3 重要性采样
- 4 信赖域
- 5 PPO
- 6 直接偏好优化 (DPO)
- 7 应用
- 8 附录：信息论基础

# 策略梯度 (REINFORCE)

← 实际中按每个 *batch* (轨迹) 更新。

REINFORCE( $s_0, \pi_\theta$ )

任意初始化  $\pi_\theta$

无限循环 (每个 episode)

使用  $\pi_\theta$  生成轨迹  $s_0, a_0, r_0, s_1, a_1, r_1, \dots, s_T, a_T, r_T$

对 episode 中每一步  $n = 0, 1, \dots, T$  循环

$$G_n \leftarrow \sum_{t=0}^{T-n} \gamma^t r_{n+t}$$

$$\text{更新策略: } \theta \leftarrow \theta + \alpha \gamma^n G_n \nabla \log \pi_\theta(a_n | s_n)$$

返回  $\pi_\theta$

\* 记号与原论文保持一致。

# 策略梯度 (REINFORCE)

← 实际中按每个 *batch* (轨迹) 更新。

REINFORCE( $s_0, \pi_\theta$ )

任意初始化  $\pi_\theta$

无限循环 (每个 episode)

使用  $\pi_\theta$  生成轨迹  $s_0, a_0, r_0, s_1, a_1, r_1, \dots, s_T, a_T, r_T$

对 episode 中每一步  $n = 0, 1, \dots, T$  循环

$$G_n \leftarrow \sum_{t=0}^{T-n} \gamma^t r_{n+t}$$

更新策略:  $\theta \leftarrow \theta + \alpha \gamma^n G_n \nabla \log \pi_\theta(a_n | s_n)$

返回  $\pi_\theta$

\* 记号与原论文保持一致。

$$\max_{\theta} J(\pi_{\theta}) \doteq \mathbb{E}_{\tau \sim \pi_{\theta}} \left[ \sum_{t=0}^{\infty} \gamma^t r_t \right]$$

$$g = \nabla_{\theta} J(\pi_{\theta}) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[ \sum_{t=0}^{\infty} \gamma^t \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) A^{\pi_{\theta}}(s_t, a_t) \right]$$

# 策略梯度定理（推导，1/2）：log-derivative 技巧

设轨迹  $\tau = (s_0, a_0, s_1, a_1, \dots)$  的概率为

$$p_{\theta}(\tau) = \rho_0(s_0) \prod_{t \geq 0} \pi_{\theta}(a_t \mid s_t) p(s_{t+1} \mid s_t, a_t), \quad R(\tau) = \sum_{t \geq 0} \gamma^t r_t.$$

# 策略梯度定理（推导，1/2）：log-derivative 技巧

设轨迹  $\tau = (s_0, a_0, s_1, a_1, \dots)$  的概率为

$$p_{\theta}(\tau) = \rho_0(s_0) \prod_{t \geq 0} \pi_{\theta}(a_t \mid s_t) p(s_{t+1} \mid s_t, a_t), \quad R(\tau) = \sum_{t \geq 0} \gamma^t r_t.$$

**第 1 步.** 对数求导技巧：  $\nabla_{\theta} p_{\theta}(\tau) = p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau)$ 。

$$\nabla_{\theta} J(\theta) = \nabla_{\theta} \int p_{\theta}(\tau) R(\tau) d\tau = \int p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau) R(\tau) d\tau = \mathbb{E}_{\tau \sim \pi_{\theta}}[R(\tau) \nabla_{\theta} \log p_{\theta}(\tau)].$$

# 策略梯度定理（推导，1/2）：log-derivative 技巧

设轨迹  $\tau = (s_0, a_0, s_1, a_1, \dots)$  的概率为

$$p_{\theta}(\tau) = \rho_0(s_0) \prod_{t \geq 0} \pi_{\theta}(a_t | s_t) p(s_{t+1} | s_t, a_t), \quad R(\tau) = \sum_{t \geq 0} \gamma^t r_t.$$

**第 1 步.** 对数求导技巧：  $\nabla_{\theta} p_{\theta}(\tau) = p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau)$ 。

$$\nabla_{\theta} J(\theta) = \nabla_{\theta} \int p_{\theta}(\tau) R(\tau) d\tau = \int p_{\theta}(\tau) \nabla_{\theta} \log p_{\theta}(\tau) R(\tau) d\tau = \mathbb{E}_{\tau \sim \pi_{\theta}} [R(\tau) \nabla_{\theta} \log p_{\theta}(\tau)].$$

**第 2 步.** 只有  $\pi_{\theta}$  依赖于  $\theta$ ，所以  $\rho_0$  和环境转移  $p(s_{t+1} | s_t, a_t)$  都被消去：

$$\nabla_{\theta} \log p_{\theta}(\tau) = \sum_{t \geq 0} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t).$$

# 策略梯度定理（推导，2/2）：因果性 + baseline

第 3 步. 利用因果性（ $t$  时刻奖励与之后的动作无关）与减去 baseline，得到

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[ \sum_{t \geq 0} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \underbrace{(Q^{\pi_{\theta}}(s_t, a_t) - b(s_t))}_{\text{当 } b=V^{\pi_{\theta}} \text{ 时即 } A^{\pi_{\theta}}(s_t, a_t)} \right].$$

# 策略梯度定理（推导，2/2）：因果性 + baseline

第 3 步. 利用因果性（ $t$  时刻奖励与之后的动作无关）与减去 baseline，得到

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[ \sum_{t \geq 0} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \underbrace{(Q^{\pi_{\theta}}(s_t, a_t) - b(s_t))}_{\text{当 } b=V^{\pi_{\theta}} \text{ 时即 } A^{\pi_{\theta}}(s_t, a_t)} \right].$$

为什么减 baseline 不影响无偏性？

$$\mathbb{E}_{a \sim \pi_{\theta}(\cdot | s)} [\nabla_{\theta} \log \pi_{\theta}(a | s) \cdot b(s)] = b(s) \nabla_{\theta} \sum_a \pi_{\theta}(a | s) = b(s) \cdot 0 = 0.$$

# 策略梯度定理（推导，2/2）：因果性 + baseline

第 3 步. 利用因果性（ $t$  时刻奖励与之后的动作无关）与减去 baseline，得到

$$\nabla_{\theta} J(\theta) = \mathbb{E}_{\tau \sim \pi_{\theta}} \left[ \sum_{t \geq 0} \nabla_{\theta} \log \pi_{\theta}(a_t | s_t) \underbrace{(Q^{\pi_{\theta}}(s_t, a_t) - b(s_t))}_{\text{当 } b=V^{\pi_{\theta}} \text{ 时即 } A^{\pi_{\theta}}(s_t, a_t)} \right].$$

为什么减 baseline 不影响无偏性？

$$\mathbb{E}_{a \sim \pi_{\theta}(\cdot | s)} [\nabla_{\theta} \log \pi_{\theta}(a | s) \cdot b(s)] = b(s) \nabla_{\theta} \sum_a \pi_{\theta}(a | s) = b(s) \cdot 0 = 0.$$

*REINFORCE* 用 *Monte-Carlo* 回报  $G_t$  替代  $A^{\pi_{\theta}}$ 。

# 目录

- 1 策略梯度
- 2 动机**
- 3 重要性采样
- 4 信赖域
- 5 PPO
- 6 直接偏好优化 (DPO)
- 7 应用
- 8 附录：信息论基础

# 存在哪些问题？(1/2)：更新不稳定

## 更新不稳定 —— 步长非常关键：

- 步长过大时：
  - 大步 → 坏策略
  - 下一批数据由当前的坏策略生成 → 采集到坏样本
  - 坏样本 → 策略更差

(对比监督学习：之后批次的正确标签和数据可以纠正它)
- 步长过小：学习过程很慢

# 存在哪些问题？(2/2)：数据效率低

## 数据效率低

- 同策略（on-policy）方法：策略一更新，就要重新采集全新轨迹
- 一次梯度更新后数据就被丢弃
- 复杂神经网络需要大量更新，这让训练过程非常缓慢

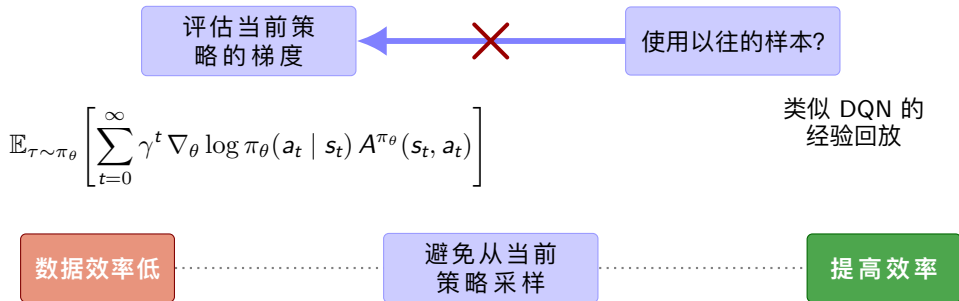
# 存在哪些问题？(2/2)：数据效率低

## 数据效率低

- 同策略 (on-policy) 方法：策略一更新，就要重新采集全新轨迹
- 一次梯度更新后数据就被丢弃
- 复杂神经网络需要大量更新，这让训练过程非常缓慢

→ 下一节我们用**重要性采样**解决“复用旧样本”的问题，  
再用 ***KL / Trust Region / 裁剪*** 解决“更新不稳定”的问题。

# 数据效率低



能否不从某个分布采样，却估计它的期望？

# 目录

- 1 策略梯度
- 2 动机
- 3 重要性采样**
- 4 信赖域
- 5 PPO
- 6 直接偏好优化 (DPO)
- 7 应用
- 8 附录：信息论基础

# 重要性采样

通过对另一个分布采样来估计某个分布

$$\mathbb{E}_{x \sim p}[f(x)] \approx \frac{1}{N} \sum_{i=1, x^i \in p}^N f(x^i)$$

$$\begin{aligned}\mathbb{E}_{x \sim p}[f(x)] &= \int f(x) p(x) dx \\ &= \int f(x) \frac{p(x)}{q(x)} q(x) dx \\ &= \mathbb{E}_{x \sim q} \left[ f(x) \frac{p(x)}{q(x)} \right]\end{aligned}$$

# 重要性采样

通过对另一个分布采样来估计某个分布

$$\mathbb{E}_{x \sim p}[f(x)] \approx \frac{1}{N} \sum_{i=1, x^i \in p}^N f(x^i)$$

$$\mathbb{E}_{x \sim p}[f(x)] = \int f(x) p(x) dx$$

$$= \mathbb{E}_{x \sim q} \left[ f(x) \frac{p(x)}{q(x)} \right]$$

通过对另一个分布采样来估计某个分布

$$\mathbb{E}_{x \sim p}[f(x)] \approx \frac{1}{N} \sum_{i=1, x^i \in p}^N f(x^i)$$

$$\begin{aligned}\mathbb{E}_{x \sim p}[f(x)] &= \int f(x) p(x) dx \\ &= \int f(x) \frac{p(x)}{q(x)} q(x) dx \\ &= \mathbb{E}_{x \sim q} \left[ f(x) \frac{p(x)}{q(x)} \right]\end{aligned}$$

# 策略梯度中的重要性采样

$$\mathbb{E}_{x \sim p}[f(x)] = \mathbb{E}_{x \sim q} \left[ f(x) \frac{p(x)}{q(x)} \right]$$

$$\begin{aligned} \nabla J(\theta) &= \mathbb{E}_{(s_t, a_t) \sim \pi_\theta} [\nabla \log \pi_\theta(a_t | s_t) A(s_t, a_t)] \\ &= \mathbb{E}_{(s_t, a_t) \sim \pi_{\theta_{\text{old}}}} \left[ \frac{\pi_\theta(s_t, a_t)}{\pi_{\theta_{\text{old}}}(s_t, a_t)} \nabla \log \pi_\theta(a_t | s_t) A(s_t, a_t) \right] \end{aligned}$$

$$J(\theta) = \mathbb{E}_{(s_t, a_t) \sim \pi_{\theta_{\text{old}}}} \left[ \frac{\pi_\theta(s_t, a_t)}{\pi_{\theta_{\text{old}}}(s_t, a_t)} A(s_t, a_t) \right] \implies \text{代理目标函数}$$

# 策略梯度中的重要性采样

$$\mathbb{E}_{x \sim p}[f(x)] = \mathbb{E}_{x \sim q} \left[ f(x) \frac{p(x)}{q(x)} \right]$$

$$\begin{aligned} \nabla J(\theta) &= \mathbb{E}_{(s_t, a_t) \sim \pi_\theta} [\nabla \log \pi_\theta(a_t | s_t) A(s_t, a_t)] \\ &= \mathbb{E}_{(s_t, a_t) \sim \pi_{\theta_{\text{old}}}} \left[ \frac{\pi_\theta(s_t, a_t)}{\pi_{\theta_{\text{old}}}(s_t, a_t)} \nabla \log \pi_\theta(a_t | s_t) A(s_t, a_t) \right] \end{aligned}$$

# 策略梯度中的重要性采样

$$\mathbb{E}_{x \sim p}[f(x)] = \mathbb{E}_{x \sim q} \left[ f(x) \frac{p(x)}{q(x)} \right]$$

$$\begin{aligned} \nabla J(\theta) &= \mathbb{E}_{(s_t, a_t) \sim \pi_\theta} [\nabla \log \pi_\theta(a_t | s_t) A(s_t, a_t)] \\ &= \mathbb{E}_{(s_t, a_t) \sim \pi_{\theta_{\text{old}}}} \left[ \frac{\pi_\theta(s_t, a_t)}{\pi_{\theta_{\text{old}}}(s_t, a_t)} \nabla \log \pi_\theta(a_t | s_t) A(s_t, a_t) \right] \end{aligned}$$

$$J(\theta) = \mathbb{E}_{(s_t, a_t) \sim \pi_{\theta_{\text{old}}}} \left[ \frac{\pi_\theta(s_t, a_t)}{\pi_{\theta_{\text{old}}}(s_t, a_t)} A(s_t, a_t) \right] \implies \text{代理目标函数}$$

# 重要性采样

有什么问题？ 天下没有免费的午餐！

两个期望相等，但我们用的是采样去估计  
→ 方差同样重要

$$\mathbb{E}_{x \sim p}[f(x)] = \mathbb{E}_{x \sim q}\left[f(x) \frac{p(x)}{q(x)}\right]$$

$$\begin{aligned} & \text{Var}_{x \sim p}[f(x)] \\ &= \mathbb{E}_{x \sim p}[f(x)^2] - (\mathbb{E}_{x \sim p}[f(x)])^2 \end{aligned}$$

$$\text{VAR}[X] = \mathbb{E}[X^2] - (\mathbb{E}[X])^2$$

$$\text{Var}_{x \sim q}\left[f(x) \frac{p(x)}{q(x)}\right]$$

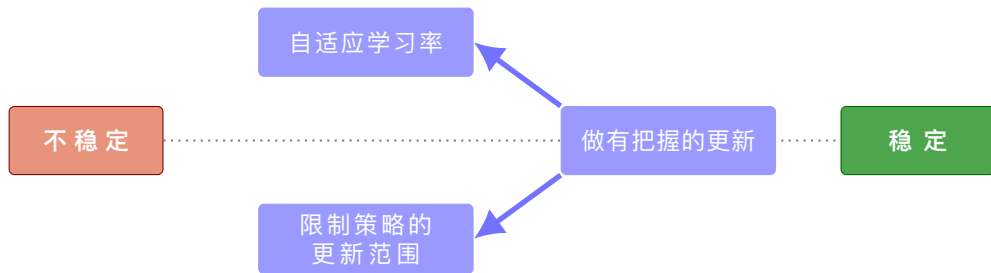
$$\begin{aligned} &= \mathbb{E}_{x \sim q}\left[\left(f(x) \frac{p(x)}{q(x)}\right)^2\right] - \left(\mathbb{E}_{x \sim q}\left[f(x) \frac{p(x)}{q(x)}\right]\right)^2 \\ &= \mathbb{E}_{x \sim p}\left[f(x)^2 \frac{p(x)}{q(x)}\right] - (\mathbb{E}_{x \sim p}[f(x)])^2 \end{aligned}$$

代价（折中）：当  $\frac{p(x)}{q(x)}$  远离 1 时，我们可能需要采集更多数据。

# 目录

- 1 策略梯度
- 2 动机
- 3 重要性采样
- 4 信赖域**
- 5 PPO
- 6 直接偏好优化 (DPO)
- 7 应用
- 8 附录：信息论基础

# 更新不稳定

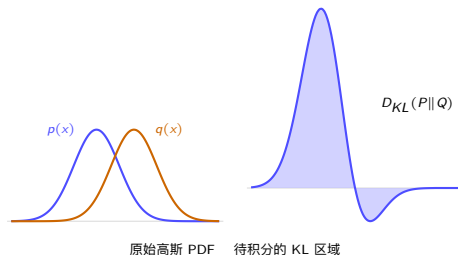


能否度量两个分布之间的距离？

# KL 散度

度量两个分布之间的距离

$$D_{KL}(P \parallel Q) = \sum_x P(x) \log \frac{P(x)}{Q(x)}$$



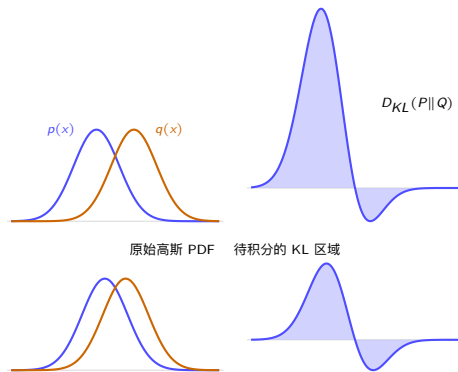
# KL 散度

度量两个分布之间的距离

$$D_{KL}(P \parallel Q) = \sum_x P(x) \log \frac{P(x)}{Q(x)}$$

两个策略之间的 KL 散度

$$D_{KL}(\pi_1 \parallel \pi_2)[s] = \sum_{a \in A} \pi_1(a \mid s) \log \frac{\pi_1(a \mid s)}{\pi_2(a \mid s)}$$



# 信赖域策略优化 (TRPO)

$$\begin{aligned} & \underset{\theta}{\text{maximize}} \quad \hat{\mathbb{E}}_t \left[ \frac{\pi_{\theta}(\mathbf{a}_t \mid \mathbf{s}_t)}{\pi_{\theta_{\text{old}}}(\mathbf{a}_t \mid \mathbf{s}_t)} \hat{A}_t \right] \\ & \text{subject to} \quad \hat{\mathbb{E}}_t [\text{KL}[\pi_{\theta_{\text{old}}}(\cdot \mid \mathbf{s}_t), \pi_{\theta}(\cdot \mid \mathbf{s}_t)]] \leq \delta \end{aligned}$$

# 信赖域策略优化 (TRPO)

$$\begin{aligned} & \underset{\theta}{\text{maximize}} \quad \hat{\mathbb{E}}_t \left[ \frac{\pi_{\theta}(\mathbf{a}_t \mid \mathbf{s}_t)}{\pi_{\theta_{\text{old}}}(\mathbf{a}_t \mid \mathbf{s}_t)} \hat{A}_t \right] \\ & \text{subject to} \quad \hat{\mathbb{E}}_t [\text{KL}[\pi_{\theta_{\text{old}}}(\cdot \mid \mathbf{s}_t), \pi_{\theta}(\cdot \mid \mathbf{s}_t)]] \leq \delta \end{aligned}$$

优化中的常见技巧：拉格朗日对偶

$$\underset{\theta}{\text{maximize}} \quad \hat{\mathbb{E}}_t \left[ \frac{\pi_{\theta}(\mathbf{a}_t \mid \mathbf{s}_t)}{\pi_{\theta_{\text{old}}}(\mathbf{a}_t \mid \mathbf{s}_t)} \hat{A}_t - \beta \text{KL}[\pi_{\theta_{\text{old}}}(\cdot \mid \mathbf{s}_t), \pi_{\theta}(\cdot \mid \mathbf{s}_t)] \right]$$

# 信赖域策略优化 (TRPO)

$$\begin{aligned} & \underset{\theta}{\text{maximize}} \quad \hat{\mathbb{E}}_t \left[ \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \hat{A}_t \right] \\ & \text{subject to} \quad \hat{\mathbb{E}}_t [\text{KL}[\pi_{\theta_{\text{old}}}(\cdot | s_t), \pi_{\theta}(\cdot | s_t)]] \leq \delta \end{aligned}$$

优化中的常见技巧：拉格朗日对偶

$$\underset{\theta}{\text{maximize}} \quad \hat{\mathbb{E}}_t \left[ \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \hat{A}_t - \beta \text{KL}[\pi_{\theta_{\text{old}}}(\cdot | s_t), \pi_{\theta}(\cdot | s_t)] \right]$$

TRPO 选择使用硬约束而不是惩罚，是因为很难选到一个  $\beta$  在不同问题——甚至在同一个问题的不同训练阶段——都能表现良好。

# 目录

- 1 策略梯度
- 2 动机
- 3 重要性采样
- 4 信赖域
- 5 PPO**
- 6 直接偏好优化 (DPO)
- 7 应用
- 8 附录：信息论基础

# 近端策略优化 (PPO)

- TRPO 用共轭梯度下降来处理约束。
- Hessian 矩阵  $\rightarrow$  计算和存储开销都很大。

# 近端策略优化 (PPO)

- TRPO 用共轭梯度下降来处理约束。
- Hessian 矩阵  $\rightarrow$  计算和存储开销都很大。

## 思路：

约束确实有助于训练过程。但也许约束并不必那么严格：

偶尔违反几次约束又有什么大碍呢？

# 近端策略优化 (PPO)

- TRPO 用共轭梯度下降来处理约束。
- Hessian 矩阵  $\rightarrow$  计算和存储开销都很大。

## 思路：

约束确实有助于训练过程。但也许约束并不必那么严格：

偶尔违反几次约束又有什么大碍呢？

把它当作“软”约束如何？在目标函数中加入一个近端项？

# 带自适应 KL 惩罚的 PPO

$$L^{KL PEN}(\theta) = \hat{\mathbb{E}}_t \left[ \frac{\pi_{\theta}(\mathbf{a}_t | \mathbf{s}_t)}{\pi_{\theta_{\text{old}}}(\mathbf{a}_t | \mathbf{s}_t)} \hat{A}_t - \beta \text{KL}[\pi_{\theta_{\text{old}}}(\cdot | \mathbf{s}_t), \pi_{\theta}(\cdot | \mathbf{s}_t)] \right]$$

# 带自适应 KL 惩罚的 PPO

$$L^{KL PEN}(\theta) = \hat{\mathbb{E}}_t \left[ \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \hat{A}_t - \beta \text{KL}[\pi_{\theta_{\text{old}}}(\cdot | s_t), \pi_{\theta}(\cdot | s_t)] \right]$$

$\beta$  不好挑  $\rightarrow$  使用自适应  $\beta$

计算  $d = \hat{\mathbb{E}}_t[\text{KL}[\pi_{\theta_{\text{old}}}(\cdot | s_t), \pi_{\theta}(\cdot | s_t)]]$

- 若  $d < d_{\text{targ}}/1.5$ ,  $\beta \leftarrow \beta/2$
- 若  $d > d_{\text{targ}} \times 1.5$ ,  $\beta \leftarrow \beta \times 2$

# 带自适应 KL 惩罚的 PPO

$$L^{KL PEN}(\theta) = \hat{\mathbb{E}}_t \left[ \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \hat{A}_t - \beta \text{KL}[\pi_{\theta_{\text{old}}}(\cdot | s_t), \pi_{\theta}(\cdot | s_t)] \right]$$

$\beta$  不好挑  $\rightarrow$  使用自适应  $\beta$

计算  $d = \hat{\mathbb{E}}_t[\text{KL}[\pi_{\theta_{\text{old}}}(\cdot | s_t), \pi_{\theta}(\cdot | s_t)]]$

- 若  $d < d_{\text{targ}}/1.5$ ,  $\beta \leftarrow \beta/2$
- 若  $d > d_{\text{targ}} \times 1.5$ ,  $\beta \leftarrow \beta \times 2$

不过仍然需要设定一个 KL 散度目标值 ...

# 带裁剪目标的 PPO

$$\underset{\theta}{\text{maximize}} \quad \hat{\mathbb{E}}_t \left[ \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \hat{A}_t \right]$$

$$r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$$

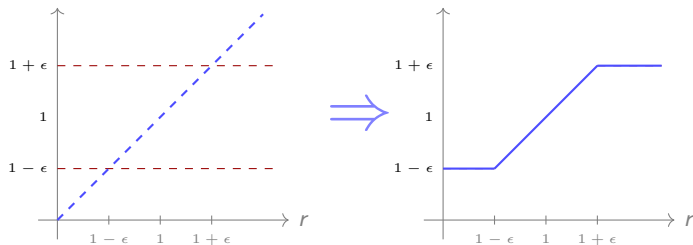
$r$  变化太快时会失去单调改进性  $\rightarrow$  把  $r$  限制在一个区间内?

# 带裁剪目标的 PPO

$$\underset{\theta}{\text{maximize}} \quad \mathbb{E}_t \left[ \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \hat{A}_t \right]$$

$$r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$$

$r$  变化太快时会失去单调改进性  $\rightarrow$  把  $r$  限制在一个区间内?

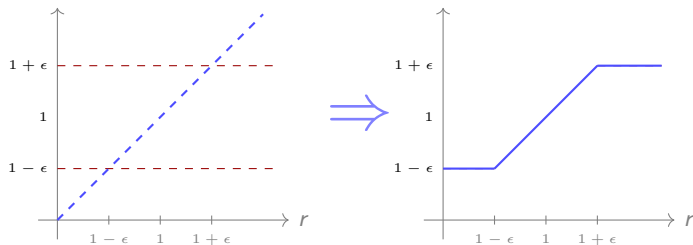


# 带裁剪目标的 PPO

$$\underset{\theta}{\text{maximize}} \quad \hat{\mathbb{E}}_t \left[ \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)} \hat{A}_t \right]$$

$$r_t(\theta) = \frac{\pi_{\theta}(a_t | s_t)}{\pi_{\theta_{\text{old}}}(a_t | s_t)}$$

$r$  变化太快时会失去单调改进性  $\rightarrow$  把  $r$  限制在一个区间内?



$$L^{CLIP}(\theta) = \hat{\mathbb{E}}_t \left[ \min(r_t(\theta) \hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A}_t) \right]$$

$$L_t^{CLIP+VF+S}(\theta) = \hat{\mathbb{E}}_t[L_t^{CLIP}(\theta) - c_1 L_t^{VF}(\theta) + c_2 S[\pi_\theta](s_t)]$$

$$L_t^{CLIP+VF+S}(\theta) = \hat{\mathbb{E}}_t[L_t^{CLIP}(\theta) - c_1 L_t^{VF}(\theta) + c_2 S[\pi_\theta](s_t)]$$

代理目标函数

critic 的平方误差损失  
 $(V_\theta(s_t) - V_t^{\text{targ}})^2$

熵奖励：保证充分探索  
鼓励”多样性”

\*  $c_1, c_2$  是经验值；原论文中  $c_1 = 1$ ,  $c_2 = 0.01$ 。

# PPO-Clip 算法

PPO-CLIP (Schulman 等, 2017)

初始化策略参数  $\theta_0$  与价值参数  $\phi_0$

**for** 迭代  $k = 0, 1, 2, \dots$  **do**

**for** actor  $= 1, 2, \dots, N$  (并行) **do**

        用策略  $\pi_{\theta_k}$  在环境中运行  $T$  个时间步

        计算优势估计  $\hat{A}_1, \dots, \hat{A}_T$  (例如 GAE)

**end for**

    令  $\theta_{\text{old}} \leftarrow \theta_k$

**for** epoch  $= 1, 2, \dots, K$  **do**

**for** 从  $NT$  个样本中取出大小为  $M$  的 minibatch **do**

$\theta \leftarrow \theta + \alpha \nabla_{\theta} L_t^{\text{CLIP} + \text{VF} + \text{S}}(\theta)$

$\phi \leftarrow \phi - \alpha \nabla_{\phi} (V_{\phi}(s_t) - V_t^{\text{targ}})^2$

**end for**

**end for**

$\theta_{k+1} \leftarrow \theta$

**end for**

# 目录

- 1 策略梯度
- 2 动机
- 3 重要性采样
- 4 信赖域
- 5 PPO
- 6 直接偏好优化 (DPO)**
- 7 应用
- 8 附录：信息论基础

# 从 PPO 到 DPO: 为什么?

## 标准 RLHF 流水线 (3 阶段):

- ① 用示范数据做监督微调 (SFT)  $\rightarrow \pi^{\text{SFT}}$
- ② 用人类偏好数据  $(x, y_w, y_l)$  训练奖励模型  $r_\phi(x, y)$
- ③ 用 PPO 微调  $\pi_\theta$ , 最大化  $\mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta} [r_\phi(x, y)] - \beta D_{KL}[\pi_\theta(\cdot | x) \| \pi_{\text{ref}}(\cdot | x)]$

# 从 PPO 到 DPO: 为什么?

## 标准 RLHF 流水线 (3 阶段):

- ① 用示范数据做监督微调 (SFT)  $\rightarrow \pi^{\text{SFT}}$
- ② 用人类偏好数据  $(x, y_w, y_l)$  训练奖励模型  $r_\phi(x, y)$
- ③ 用 PPO 微调  $\pi_\theta$ , 最大化  $\mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta} [r_\phi(x, y)] - \beta D_{\text{KL}}[\pi_\theta(\cdot | x) \parallel \pi_{\text{ref}}(\cdot | x)]$

## 这套流程中 PPO 的问题:

- 在 SFT 之上还要多两个训练阶段 (奖励模型 + 强化学习)
- PPO 在大模型上不稳定、显存压力大、超参敏感
- 奖励模型可能被“奖励攻击”或建模错误

# 从 PPO 到 DPO: 为什么?

## 标准 RLHF 流水线 (3 阶段):

- ① 用示范数据做监督微调 (SFT)  $\rightarrow \pi^{\text{SFT}}$
- ② 用人类偏好数据  $(x, y_w, y_l)$  训练奖励模型  $r_\phi(x, y)$
- ③ 用 PPO 微调  $\pi_\theta$ , 最大化  $\mathbb{E}_{x \sim \mathcal{D}, y \sim \pi_\theta} [r_\phi(x, y)] - \beta D_{\text{KL}}[\pi_\theta(\cdot | x) \parallel \pi_{\text{ref}}(\cdot | x)]$

## 这套流程中 PPO 的问题:

- 在 SFT 之上还要多两个训练阶段 (奖励模型 + 强化学习)
- PPO 在大模型上不稳定、显存压力大、超参敏感
- 奖励模型可能被“奖励攻击”或建模错误

能否直接从偏好数据优化策略——完全跳过训练奖励模型?

# DPO 推导 (1/5): KL 目标与拉格朗日

KL 正则化目标, 展开形式:

$$\max_{\pi} \mathbb{E}_{x \sim \mathcal{D}} \mathbb{E}_{y \sim \pi(\cdot|x)} \left[ r(x, y) - \beta \log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} \right]$$

# DPO 推导 (1/5): KL 目标与拉格朗日

KL 正则化目标, 展开形式:

$$\max_{\pi} \mathbb{E}_{x \sim \mathcal{D}} \mathbb{E}_{y \sim \pi(\cdot|x)} \left[ r(x, y) - \beta \log \frac{\pi(y|x)}{\pi_{\text{ref}}(y|x)} \right]$$

逐 prompt 求解 (固定  $x$ ); 为  $\sum_y \pi(y|x) = 1$  引入拉格朗日乘子  $\lambda$ :

$$\mathcal{L} = \sum_y \pi(y|x) [r(x, y) + \beta \log \pi_{\text{ref}}(y|x) - \beta \log \pi(y|x)] - \lambda \left( \sum_y \pi(y|x) - 1 \right)$$

## DPO 推导 (2/5): KKT 与闭式解

**KKT 稳定性条件**,  $\partial \mathcal{L} / \partial \pi(y | x) = 0$ :

$$r(x, y) + \beta \log \pi_{\text{ref}}(y | x) - \beta \log \pi(y | x) - \beta - \lambda = 0$$

$$\implies \log \pi(y | x) = \log \pi_{\text{ref}}(y | x) + \frac{1}{\beta} r(x, y) - \frac{\beta + \lambda}{\beta}.$$

## DPO 推导 (2/5): KKT 与闭式解

**KKT 稳定性条件**,  $\partial \mathcal{L} / \partial \pi(y | x) = 0$ :

$$r(x, y) + \beta \log \pi_{\text{ref}}(y | x) - \beta \log \pi(y | x) - \beta - \lambda = 0$$

$$\implies \log \pi(y | x) = \log \pi_{\text{ref}}(y | x) + \frac{1}{\beta} r(x, y) - \frac{\beta + \lambda}{\beta}.$$

**取指数并归一化** ( $\lambda$  被吸收进  $Z(x)$ ):

$$\pi^*(y | x) = \frac{1}{Z(x)} \pi_{\text{ref}}(y | x) \exp\left(\frac{1}{\beta} r(x, y)\right), \quad Z(x) = \sum_y \pi_{\text{ref}}(y | x) \exp\left(\frac{1}{\beta} r(x, y)\right).$$

## DPO 推导 (3/5): 奖励反解与 Bradley–Terry

反解闭式  $\pi^*$ , 把  $r$  用策略表示:

$$r(x, y) = \beta \log \frac{\pi^*(y | x)}{\pi_{\text{ref}}(y | x)} + \beta \log Z(x).$$

# DPO 推导 (3/5): 奖励反解与 Bradley–Terry

反解闭式  $\pi^*$ , 把  $r$  用策略表示:

$$r(x, y) = \beta \log \frac{\pi^*(y | x)}{\pi_{\text{ref}}(y | x)} + \beta \log Z(x).$$

Bradley–Terry 偏好模型:

$$P(y_w \succ y_l | x) = \sigma(r(x, y_w) - r(x, y_l)).$$

# DPO 推导 (3/5): 奖励反解与 Bradley-Terry

反解闭式  $\pi^*$ , 把  $r$  用策略表示:

$$r(x, y) = \beta \log \frac{\pi^*(y | x)}{\pi_{\text{ref}}(y | x)} + \beta \log Z(x).$$

Bradley-Terry 偏好模型:

$$P(y_w \succ y_l | x) = \sigma(r(x, y_w) - r(x, y_l)).$$

下一页: 代入隐式奖励, 看  $Z(x)$  怎么消失。

## DPO 推导 (4/5): $Z(x)$ 抵消 & DPO 损失

代入隐式奖励，看一看差值：

$$\begin{aligned} r(x, y_w) - r(x, y_l) &= \beta \log \frac{\pi^*(y_w | x)}{\pi_{\text{ref}}(y_w | x)} + \cancel{\beta \log Z(x)} - \beta \log \frac{\pi^*(y_l | x)}{\pi_{\text{ref}}(y_l | x)} - \cancel{\beta \log Z(x)} \\ &= \beta \log \frac{\pi^*(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi^*(y_l | x)}{\pi_{\text{ref}}(y_l | x)}. \end{aligned}$$

难以计算的  $Z(x)$  在两个隐式奖励的差值中消失了。

## DPO 推导 (4/5): $Z(x)$ 抵消 & DPO 损失

代入隐式奖励，看一看差值：

$$\begin{aligned} r(x, y_w) - r(x, y_l) &= \beta \log \frac{\pi^*(y_w | x)}{\pi_{\text{ref}}(y_w | x)} + \cancel{\beta \log Z(x)} - \beta \log \frac{\pi^*(y_l | x)}{\pi_{\text{ref}}(y_l | x)} - \cancel{\beta \log Z(x)} \\ &= \beta \log \frac{\pi^*(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi^*(y_l | x)}{\pi_{\text{ref}}(y_l | x)}. \end{aligned}$$

难以计算的  $Z(x)$  在两个隐式奖励的差值中消失了。

对偏好数据  $(x, y_w, y_l) \sim \mathcal{D}$  做最大似然：

$$\mathcal{L}_{\text{DPO}}(\theta) = -\mathbb{E}_{\mathcal{D}} \left[ \log \sigma \left( \beta \log \frac{\pi_{\theta}(y_w | x)}{\pi_{\text{ref}}(y_w | x)} - \beta \log \frac{\pi_{\theta}(y_l | x)}{\pi_{\text{ref}}(y_l | x)} \right) \right]$$

# DPO 推导 (5/5): 梯度与解释

定义隐式奖励:  $\hat{r}_\theta(x, y) \doteq \beta \log \frac{\pi_\theta(y | x)}{\pi_{\text{ref}}(y | x)}.$

## DPO 推导 (5/5): 梯度与解释

定义隐式奖励:  $\hat{r}_\theta(x, y) \doteq \beta \log \frac{\pi_\theta(y | x)}{\pi_{\text{ref}}(y | x)}$ .

对 DPO 损失求导 (利用  $\frac{d}{du} \log \sigma(u) = \sigma(-u)$ ):

$$\begin{aligned}\nabla_\theta \mathcal{L}_{\text{DPO}}(\theta) &= -\mathbb{E}_{\mathcal{D}} \left[ \sigma(\hat{r}_\theta(x, y_l) - \hat{r}_\theta(x, y_w)) \nabla_\theta (\hat{r}_\theta(x, y_w) - \hat{r}_\theta(x, y_l)) \right] \\ &= -\beta \mathbb{E}_{\mathcal{D}} \left[ \underbrace{\sigma(\hat{r}_\theta(x, y_l) - \hat{r}_\theta(x, y_w))}_{\text{权重: 模型当前错得有多严重}} \cdot \left( \nabla_\theta \log \pi_\theta(y_w | x) - \nabla_\theta \log \pi_\theta(y_l | x) \right) \right].\end{aligned}$$

# DPO 推导 (5/5): 梯度与解释

定义隐式奖励:  $\hat{r}_\theta(x, y) \doteq \beta \log \frac{\pi_\theta(y | x)}{\pi_{\text{ref}}(y | x)}$ .

对 DPO 损失求导 (利用  $\frac{d}{du} \log \sigma(u) = \sigma(-u)$ ):

$$\begin{aligned}\nabla_\theta \mathcal{L}_{\text{DPO}}(\theta) &= -\mathbb{E}_{\mathcal{D}} \left[ \sigma(\hat{r}_\theta(x, y_l) - \hat{r}_\theta(x, y_w)) \nabla_\theta (\hat{r}_\theta(x, y_w) - \hat{r}_\theta(x, y_l)) \right] \\ &= -\beta \mathbb{E}_{\mathcal{D}} \left[ \underbrace{\sigma(\hat{r}_\theta(x, y_l) - \hat{r}_\theta(x, y_w))}_{\text{权重: 模型当前错得有多严重}} \cdot \left( \nabla_\theta \log \pi_\theta(y_w | x) - \nabla_\theta \log \pi_\theta(y_l | x) \right) \right].\end{aligned}$$

## 解释

- 权重  $\sigma(\hat{r}_l - \hat{r}_w)$  在模型仍偏好  $y_l$  而非  $y_w$  时 (即  $\hat{r}_l > \hat{r}_w$ ) 较大; 偏好顺序正确时趋近 0。
- 方向  $\nabla \log \pi_\theta(y_w) - \nabla \log \pi_\theta(y_l) =$  提升  $y_w$ 、压低  $y_l$ 。
- 纯一阶优化; 显存开销与监督微调相同。

# PPO 与 DPO 对比 (1/2): 对比表

|                     | 基于 PPO 的 RLHF                                | DPO                             |
|---------------------|----------------------------------------------|---------------------------------|
| 奖励模型                | 单独训练 $r_\phi$                                | 隐式 (不训练)                        |
| SFT 之后的阶段数          | 2 (RM + RL)                                  | 1 (仅 DPO)                       |
| 在线从 $\pi_\theta$ 采样 | 需要 (rollout)                                 | 不需要 (使用静态偏好)                    |
| 优化方式                | 策略梯度 + 比值裁剪                                  | 对 log-ratio 做监督式 NLL            |
| 稳定性                 | 敏感 (clip、 $\beta$ 、KL 目标)                    | 明显更稳定                           |
| 显存                  | $\pi_\theta + \pi_{\text{old}} + r_\phi + V$ | $\pi_\theta + \pi_{\text{ref}}$ |
| 单步计算量               | 生成 + RM 评分 + PG 更新                           | 2 次前向 + 1 次反向                   |
| 探索                  | 有 (采样)                                       | 无 (受限于数据)                       |

# PPO 与 DPO 对比 (2/2): 解读

- DPO 用探索能力换取简洁与稳定：
  - 没有 rollout, 所以训练曲线像 SFT 一样平稳;
  - 但模型也只能在数据集已经覆盖的区域上学习。
- DPO 的质量上限受偏好数据的质量与覆盖度限制：
  - 偏好对里如果不包含某种回答模式, DPO 无从学起;
  - 这也是后续 iterative/online DPO 等变种的动机。
- 当存在新鲜的 on-policy 样本或真实奖励信号时 (游戏、机器人、有真奖励的 LLM 任务), **PPO / 在线 RL 仍然占优。**

# 目录

- 1 策略梯度
- 2 动机
- 3 重要性采样
- 4 信赖域
- 5 PPO
- 6 直接偏好优化 (DPO)
- 7 应用**
- 8 附录：信息论基础

# PPO 的应用 (1/2): 游戏 & 机器人

## 游戏

- **OpenAI Five** (Dota 2, 2018–19): 击败世界冠军; PPO 扩展到 128,000 CPU 核。
- Atari、ProcGen、MineRL 基准。
- AlphaStar 类似工作 (星际争霸 II) 的基线。

## 机器人与控制

- MuJoCo / DM Control (Humanoid、Ant、Walker)。
- OpenAI *Dactyl*——单手魔方操控。
- 四足运动控制 (ANYmal、MIT Mini Cheetah), 含仿真到现实迁移。
- 自动驾驶仿真器 (CARLA)。

# PPO 的应用 (2/2): 大模型对齐 & 其他

## 大模型对齐 (RLHF)

- **InstructGPT** / ChatGPT / GPT-4。
- Anthropic 早期的 Claude (Constitutional AI 也用 PPO)。
- Llama 2-Chat、Gemini、Qwen-Chat。

## 其他

- 芯片布局 (Google, 2021)。
- 推荐系统、广告竞价。
- 科学发现 (矩阵乘法、AlphaTensor 类)。

**常用库:** stable-baselines3、CleanRL、RLlib、TRL。

# DPO 的应用 (1/2): 开源模型 & 应用领域

## 开源大模型对齐

- **Zephyr-7B** (HuggingFace H4) ——最早被广泛引用的 DPO 模型。
- **Tulu 2** (AI2)、**Notus**、**Starling**。
- **Llama 3-Instruct** (Meta, 2024) ——官方流水线使用 DPO + 拒绝采样。
- **Mistral / Mixtral** 的 chat 版本。
- **Qwen 2 / Qwen 2.5** (阿里) ——后训练阶段使用 DPO。

## 应用领域

- 指令跟随、有用性、安全性。
- 代码生成 (CodeLlama-DPO 等)。
- 数学与推理 (对思维链做偏好)。
- 摘要、翻译。

# DPO 的应用 (2/2): 变种 & 常用库

## 重要的 DPO 变种

- **IPO**——去除 Bradley-Terry 假设。
- **KTO**——单样本（不需要成对偏好），基于前景理论。
- **ORPO**——把 SFT 和偏好损失合并到一个阶段。
- **cDPO / rDPO**——对噪声标签更鲁棒。
- **迭代 / 在线 DPO**——自评分、生成-再偏好循环。
- **Step-DPO**——对推理步骤做偏好。

**常用库：** HuggingFace TRL、Axolotl、LLaMA-Factory、OpenRLHF。

# 总结 (1/2): 经典策略梯度

- **REINFORCE**——简单、无偏，但同策略、方差大、样本效率低。
- **重要性采样 (IS)** ——允许复用异策略样本；代价是当  $\pi_{\theta}/\pi_{\theta_{\text{old}}}$  远离 1 时方差变大。
- **TRPO**——硬 KL 约束保证单调改进；但需要共轭梯度与 Hessian 向量积，工程上较重。

## 总结 (2/2): PPO 与 DPO

- **PPO-KL**——带自适应  $\beta$  的软 KL 惩罚；比 TRPO 简单，但仍需指定目标 KL。
- **PPO-Clip**——把 KL 换成裁剪后的比值  $r_t(\theta)$ ；纯一阶、多 epoch 复用，是当今 on-policy 深度 RL 的事实基线。
- **DPO**——完全跳过奖励模型：在 log-ratio  $\pi_\theta / \pi_{\text{ref}}$  上做监督式损失；理论上与 KL 正则化的 RLHF 等价，但工程上简单很多，已成为开源 LLM 后训练的常用方案。

- ① Schulman, J., Wolski, F., Dhariwal, P., Radford, A., & Klimov, O. (2017). *Proximal Policy Optimization Algorithms*. arXiv:1707.06347.
- ② Rafailov, R., Sharma, A., Mitchell, E., Ermon, S., Manning, C. D., & Finn, C. (2023). *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*. NeurIPS.
- ③ Ouyang, L., Wu, J., Jiang, X., et al. (2022). *Training language models to follow instructions with human feedback (InstructGPT)*. NeurIPS.
- ④ Schulman, J., Levine, S., Moritz, P., Jordan, M., & Abbeel, P. (2015). *Trust Region Policy Optimization*. ICML.
- ⑤ Schulman, J., Moritz, P., Levine, S., Jordan, M., & Abbeel, P. (2016). *High-Dimensional Continuous Control Using Generalized Advantage Estimation*. ICLR.
- ⑥ Williams, R. J. (1992). *Simple statistical gradient-following algorithms for connectionist reinforcement learning*. Machine Learning, 8(3–4), 229–256.
- ⑦ Sutton, R. S., & Barto, A. G. (2018). *Reinforcement Learning: An Introduction* (2nd ed.). MIT Press.

# 目录

- 1 策略梯度
- 2 动机
- 3 重要性采样
- 4 信赖域
- 5 PPO
- 6 直接偏好优化 (DPO)
- 7 应用
- 8 附录：信息论基础**

# 为什么要学 Entropy / Cross Entropy / KL / Mutual Information

这些概念都在衡量同一件事的不同侧面：

- 不确定性 —— entropy
- 预测代价 —— cross entropy
- 分布之间的差异 —— KL divergence
- 两个变量共享的信息 —— mutual information

# 为什么要学 Entropy / Cross Entropy / KL / Mutual Information

这些概念都在衡量同一件事的不同侧面：

- **不确定性** ——entropy
- **预测代价** ——cross entropy
- **分布之间的差异** ——KL divergence
- **两个变量共享的信息** ——mutual information

在机器学习里它们怎么用？

- **Cross entropy** 常作为分类 loss
- **KL divergence** 用于比较两个分布（PPO、DPO 里的 KL 项就是这个）
- **Mutual information** 衡量两个随机变量共享多少信息

# 为什么要学 Entropy / Cross Entropy / KL / Mutual Information

这些概念都在衡量同一件事的不同侧面：

- **不确定性** ——entropy
- **预测代价** ——cross entropy
- **分布之间的差异** ——KL divergence
- **两个变量共享的信息** ——mutual information

在机器学习里它们怎么用？

- **Cross entropy** 常作为分类 loss
- **KL divergence** 用于比较两个分布（PPO、DPO 里的 KL 项就是这个）
- **Mutual information** 衡量两个随机变量共享多少信息

我们从最简单的问题切入：常出现的符号能不能用更短的编码？  
罕见的符号能不能用更长的编码？

# 为什么要学 Entropy / Cross Entropy / KL / Mutual Information

这些概念都在衡量同一件事的不同侧面：

- **不确定性** ——entropy
- **预测代价** ——cross entropy
- **分布之间的差异** ——KL divergence
- **两个变量共享的信息** ——mutual information

在机器学习里它们怎么用？

- **Cross entropy** 常作为分类 loss
- **KL divergence** 用于比较两个分布（PPO、DPO 里的 KL 项就是这个）
- **Mutual information** 衡量两个随机变量共享多少信息

我们从最简单的问题切入：常出现的符号能不能用更短的编码？  
罕见的符号能不能用更长的编码？

**核心直觉：** 概率  $\uparrow \Rightarrow$  信息量 / 惊讶程度  $\downarrow$   
概率  $\downarrow \Rightarrow$  信息量 / 惊讶程度  $\uparrow$

# 单个事件的信息量： $-\log p$

事件  $x$  发生的概率是  $p(x)$ ，那么它的**信息量**定义为

$$I(x) = -\log_2 p(x).$$

概率越小  $\Rightarrow$  发生时越“惊讶”  $\Rightarrow$  信息量越大。

# 单个事件的信息量: $-\log p$

事件  $x$  发生的概率是  $p(x)$ , 那么它的**信息量**定义为

$$I(x) = -\log_2 p(x).$$

概率越小  $\Rightarrow$  发生时越“惊讶”  $\Rightarrow$  信息量越大。

例子

| $p$      | $I = -\log_2 p$ |
|----------|-----------------|
| $1/2$    | 1 bit           |
| $1/4$    | 2 bits          |
| $1/8$    | 3 bits          |
| $1/1024$ | 10 bits         |

# 单个事件的信息量： $-\log p$

事件  $x$  发生的概率是  $p(x)$ ，那么它的**信息量**定义为

$$I(x) = -\log_2 p(x).$$

概率越小  $\Rightarrow$  发生时越“惊讶”  $\Rightarrow$  信息量越大。

例子

| $p$    | $I = -\log_2 p$ |
|--------|-----------------|
| 1/2    | 1 bit           |
| 1/4    | 2 bits          |
| 1/8    | 3 bits          |
| 1/1024 | 10 bits         |

一个很常见的事件不会带来多少新信息；  
一个很罕见的事件发生时携带的信息量很大。

# Entropy: 分布自身的平均不确定性

对随机变量  $X \sim P$ , **entropy** 定义为

$$H(P) = H(X) = - \sum_x P(x) \log_2 P(x).$$

**两种解释:**

- 数据真的来自  $P$  时, 平均每个符号带来多少信息量;
- 最优压缩时, 平均每个符号至少需要多少 bit。

# Entropy: 分布自身的平均不确定性

对随机变量  $X \sim P$ , **entropy** 定义为

$$H(P) = H(X) = - \sum_x P(x) \log_2 P(x).$$

两种解释:

- 数据真的来自  $P$  时, 平均每个符号带来多少信息量;
- 最优压缩时, 平均每个符号至少需要多少 bit.

例子

- 公平硬币  $P(H) = P(T) = 0.5$ :

$$H(X) = -0.5 \log_2 0.5 - 0.5 \log_2 0.5 = 1 \text{ bit.}$$

- 偏置硬币  $P(H) = 0.9, P(T) = 0.1$ :

$$H(X) \approx 0.469 \text{ bit.}$$

# Entropy: 分布自身的平均不确定性

对随机变量  $X \sim P$ , **entropy** 定义为

$$H(P) = H(X) = - \sum_x P(x) \log_2 P(x).$$

两种解释:

- 数据真的来自  $P$  时, 平均每个符号带来多少信息量;
- 最优压缩时, 平均每个符号至少需要多少 bit.

例子

- 公平硬币  $P(H) = P(T) = 0.5$ :

$$H(X) = -0.5 \log_2 0.5 - 0.5 \log_2 0.5 = 1 \text{ bit.}$$

- 偏置硬币  $P(H) = 0.9, P(T) = 0.1$ :

$$H(X) \approx 0.469 \text{ bit.}$$

分布越**均匀**  $\Rightarrow$  *entropy* 越大;  
分布越**集中** (接近确定性)  $\Rightarrow$  *entropy* 越小。

# Prefix Code 与 Kraft Inequality

**Prefix code 规则：**任何 codeword 都不能是另一个 codeword 的前缀。

例如  $A \mapsto 0$ ,  $B \mapsto 10$ ,  $C \mapsto 110$ ,  $D \mapsto 111$  就是合法的 prefix code —— 解码时看到 0 立刻知道是 A, 不会和 10、110、111 混淆。

# Prefix Code 与 Kraft Inequality

**Prefix code 规则：**任何 codeword 都不能是另一个 codeword 的前缀。

例如  $A \mapsto 0$ ,  $B \mapsto 10$ ,  $C \mapsto 110$ ,  $D \mapsto 111$  就是合法的 prefix code —— 解码时看到 0 立刻知道是 A, 不会和 10、110、111 混淆。

**Kraft inequality：**若码长为  $\ell_1, \ell_2, \dots, \ell_m$ , 则合法的 prefix code 必须满足

$$\sum_i 2^{-\ell_i} \leq 1.$$

直觉：长度为  $\ell_i$  的码字“占用”了  $2^{-\ell_i}$  的前缀空间；所有码字加起来不能超过整个编码空间（1）。

# Prefix Code 与 Kraft Inequality

**Prefix code 规则：**任何 codeword 都不能是另一个 codeword 的前缀。

例如  $A \mapsto 0$ ,  $B \mapsto 10$ ,  $C \mapsto 110$ ,  $D \mapsto 111$  就是合法的 prefix code —— 解码时看到 0 立刻知道是 A, 不会和 10、110、111 混淆。

**Kraft inequality：**若码长为  $\ell_1, \ell_2, \dots, \ell_m$ , 则合法的 prefix code 必须满足

$$\sum_i 2^{-\ell_i} \leq 1.$$

直觉：长度为  $\ell_i$  的码字“占用”了  $2^{-\ell_i}$  的前缀空间；所有码字加起来不能超过整个编码空间（1）。

连接到 **entropy**：理想码长是  $\ell_i \approx -\log_2 p_i$ , 于是

$$2^{-\ell_i} = p_i, \quad \sum_i 2^{-\ell_i} = \sum_i p_i = 1.$$

这是 Kraft 的等号情形——也就是把编码空间用满。

# Huffman Coding: 高概率短码, 低概率长码

**Huffman coding** 是构造最优 prefix code 的经典方法。

目标:  $\min \sum_i p_i \ell_i$ .

算法直觉: 每次合并概率最小的两个符号, 直到形成一棵二叉树。

# Huffman Coding: 高概率短码, 低概率长码

**Huffman coding** 是构造最优 prefix code 的经典方法。

目标:  $\min \sum_i p_i \ell_i$ .

算法直觉: 每次合并概率最小的两个符号, 直到形成一棵二叉树。

例子  $P(A) = 0.5, P(B) = 0.25, P(C) = 0.15, P(D) = 0.10$ , 可得

$$A \mapsto 0, \quad B \mapsto 10, \quad C \mapsto 110, \quad D \mapsto 111.$$

平均码长:

$$L = 0.5 \times 1 + 0.25 \times 2 + 0.15 \times 3 + 0.10 \times 3 = 1.75 \text{ bit/符号}.$$

# Huffman Coding: 高概率短码, 低概率长码

**Huffman coding** 是构造最优 prefix code 的经典方法。

目标:  $\min \sum_i p_i \ell_i$ .

算法直觉: 每次合并概率最小的两个符号, 直到形成一棵二叉树。

例子  $P(A) = 0.5, P(B) = 0.25, P(C) = 0.15, P(D) = 0.10$ , 可得

$$A \mapsto 0, \quad B \mapsto 10, \quad C \mapsto 110, \quad D \mapsto 111.$$

平均码长:

$$L = 0.5 \times 1 + 0.25 \times 2 + 0.15 \times 3 + 0.10 \times 3 = 1.75 \text{ bit/符号}.$$

对应的 entropy:  $H(P) \approx 1.743 \text{ bit/符号}$ 。

*Huffman* 的平均码长非常接近 *entropy* ——  
但因为单符号码长必须是整数 *bit*, 通常无法完全相等。

# Block Coding: 把多个符号合成一个“大符号”

单个符号的码长必须是**整数 bit**，所以通常无法完全达到 entropy。

**反例**  $P(0) = 0.9, P(1) = 0.1$ :

$$H(X) \approx 0.469 \text{ bit/符号},$$

但单独编码每个符号至少需要 1 bit。

# Block Coding: 把多个符号合成一个“大符号”

单个符号的码长必须是**整数 bit**，所以通常无法完全达到 entropy。

**反例**  $P(0) = 0.9, P(1) = 0.1$ :

$$H(X) \approx 0.469 \text{ bit/符号},$$

但单独编码每个符号至少需要 1 bit。

**解决**：把相邻的 2 个符号合成一个 block：

$$P(00) = 0.81, P(01) = 0.09, P(10) = 0.09, P(11) = 0.01.$$

对 block 做 Huffman:  $00 \mapsto 0, 01 \mapsto 10, 10 \mapsto 110, 11 \mapsto 111$ 。

平均码长:  $L \approx 1.29 \text{ bit/block} = 0.645 \text{ bit/原始符号}$ 。

# Block Coding: 把多个符号合成一个“大符号”

单个符号的码长必须是**整数 bit**，所以通常无法完全达到 entropy。

**反例**  $P(0) = 0.9, P(1) = 0.1$ :

$$H(X) \approx 0.469 \text{ bit/符号},$$

但单独编码每个符号至少需要 1 bit。

**解决**: 把相邻的 2 个符号合成一个 block:

$$P(00) = 0.81, P(01) = 0.09, P(10) = 0.09, P(11) = 0.01.$$

对 block 做 Huffman:  $00 \mapsto 0, 01 \mapsto 10, 10 \mapsto 110, 11 \mapsto 111$ 。

平均码长:  $L \approx 1.29 \text{ bit/block} = 0.645 \text{ bit/原始符号}$ 。

比单独编码的 1 bit/符号更接近 entropy (0.469)。

结论: *block* 越长, 平均每个符号的编码长度越接近 *entropy*。

# Cross Entropy: 用 $Q$ 预测来自 $P$ 的数据

真实分布是  $P$ , 模型认为分布是  $Q$ 。

**Cross entropy:**

$$H(P, Q) = - \sum_x P(x) \log_2 Q(x).$$

解释: 数据真实来自  $P$ , 但我们按照  $Q$  来预测或编码, 平均代价就是  $H(P, Q)$ 。

# Cross Entropy: 用 $Q$ 预测来自 $P$ 的数据

真实分布是  $P$ , 模型认为分布是  $Q$ 。

**Cross entropy:**

$$H(P, Q) = - \sum_x P(x) \log_2 Q(x).$$

解释: 数据真实来自  $P$ , 但我们按照  $Q$  来预测或编码, 平均代价就是  $H(P, Q)$ 。

**与 entropy 的区别**

- $H(P) = - \sum_x P(x) \log_2 P(x)$ : 按真实分布  $P$  编码。
- $H(P, Q) = - \sum_x P(x) \log_2 Q(x)$ : 按模型分布  $Q$  编码真实数据。

# Cross Entropy: 用 $Q$ 预测来自 $P$ 的数据

真实分布是  $P$ , 模型认为分布是  $Q$ 。

**Cross entropy:**

$$H(P, Q) = - \sum_x P(x) \log_2 Q(x).$$

解释: 数据真实来自  $P$ , 但我们按照  $Q$  来预测或编码, 平均代价就是  $H(P, Q)$ 。

**与 entropy 的区别**

- $H(P) = - \sum_x P(x) \log_2 P(x)$ : 按真实分布  $P$  编码。
- $H(P, Q) = - \sum_x P(x) \log_2 Q(x)$ : 按模型分布  $Q$  编码真实数据。

**机器学习里的分类 loss**    真实标签 one-hot  $P = (0, 1, 0)$ , 模型输出  $Q = (0.1, 0.7, 0.2)$ :

$$H(P, Q) = -\log_2 0.7.$$

Cross entropy loss 惩罚的就是: **模型给真实类别的概率太低。**

# KL Divergence: 额外浪费的平均编码长度

**KL divergence:**

$$D_{KL}(P \parallel Q) = \sum_x P(x) \log_2 \frac{P(x)}{Q(x)}.$$

**与 entropy 和 cross entropy 的关系:**

$$H(P, Q) = H(P) + D_{KL}(P \parallel Q)$$

即  $D_{KL}(P \parallel Q) = H(P, Q) - H(P)$ 。

# KL Divergence: 额外浪费的平均编码长度

KL divergence:

$$D_{KL}(P \parallel Q) = \sum_x P(x) \log_2 \frac{P(x)}{Q(x)}.$$

与 entropy 和 cross entropy 的关系:

$$H(P, Q) = H(P) + D_{KL}(P \parallel Q)$$

即  $D_{KL}(P \parallel Q) = H(P, Q) - H(P)$ 。

编码解释

- $H(P)$ : 知道真实分布  $P$  时的最优平均码长。
- $H(P, Q)$ : 误以为分布是  $Q$ , 用  $Q$  编码真实数据时的平均码长。
- $D_{KL}(P \parallel Q)$ : 因为用了错误分布  $Q$ , 平均每个符号多浪费了多少 bit。

# KL Divergence: 额外浪费的平均编码长度

KL divergence:

$$D_{KL}(P \parallel Q) = \sum_x P(x) \log_2 \frac{P(x)}{Q(x)}.$$

与 entropy 和 cross entropy 的关系:

$$H(P, Q) = H(P) + D_{KL}(P \parallel Q)$$

即  $D_{KL}(P \parallel Q) = H(P, Q) - H(P)$ 。

编码解释

- $H(P)$ : 知道真实分布  $P$  时的最优平均码长。
- $H(P, Q)$ : 误以为分布是  $Q$ , 用  $Q$  编码真实数据时的平均码长。
- $D_{KL}(P \parallel Q)$ : 因为用了错误分布  $Q$ , 平均每个符号多浪费了多少 bit。

注意: KL divergence 通常不对称:

$$D_{KL}(P \parallel Q) \neq D_{KL}(Q \parallel P).$$

所以它不是通常意义上的“距离” (不满足对称性, 也不满足三角不等式)。

# Mutual Information: 知道 $Y$ 后 $X$ 的不确定性减少多少

## Mutual information:

$$I(X; Y) = H(X) - H(X | Y).$$

## 解释

- 原来  $X$  的不确定性是  $H(X)$ 。
- 知道  $Y$  以后,  $X$  的剩余不确定性是  $H(X | Y)$ 。
- 减少的那部分就是**互信息**  $I(X; Y)$ 。

# Mutual Information: 知道 $Y$ 后 $X$ 的不确定性减少多少

## Mutual information:

$$I(X; Y) = H(X) - H(X | Y).$$

## 解释

- 原来  $X$  的不确定性是  $H(X)$ 。
- 知道  $Y$  以后,  $X$  的剩余不确定性是  $H(X | Y)$ 。
- 减少的那部分就是**互信息**  $I(X; Y)$ 。

## 等价表达

$$I(X; Y) = H(Y) - H(Y | X) = H(X) + H(Y) - H(X, Y).$$

推论: 互信息是**对称的**,  $I(X; Y) = I(Y; X)$ 。

# Mutual Information: 知道 $Y$ 后 $X$ 的不确定性减少多少

## Mutual information:

$$I(X; Y) = H(X) - H(X | Y).$$

### 解释

- 原来  $X$  的不确定性是  $H(X)$ 。
- 知道  $Y$  以后,  $X$  的剩余不确定性是  $H(X | Y)$ 。
- 减少的那部分就是**互信息**  $I(X; Y)$ 。

### 等价表达

$$I(X; Y) = H(Y) - H(Y | X) = H(X) + H(Y) - H(X, Y).$$

推论: 互信息是**对称的**,  $I(X; Y) = I(Y; X)$ 。

### 直觉

- $X, Y$  独立  $\Rightarrow$  知道  $Y$  对预测  $X$  没帮助  $\Rightarrow I(X; Y) = 0$ 。
- $Y$  能很好预测  $X \Rightarrow$  互信息很大。

# Mutual Information 是 KL Divergence 的特殊情况

互信息可以写成一个 KL divergence:

$$I(X; Y) = D_{KL}(P_{XY} \parallel P_X P_Y)$$

其中:

- $P_{XY}$  是真实的 **joint distribution**;
- $P_X P_Y$  是假设  $X, Y$  独立时的 joint distribution。

# Mutual Information 是 KL Divergence 的特殊情况

互信息可以写成一个 KL divergence:

$$I(X; Y) = D_{KL}(P_{XY} \parallel P_X P_Y)$$

其中:

- $P_{XY}$  是真实的 **joint distribution**;
- $P_X P_Y$  是假设  $X, Y$  独立时的 joint distribution。

两种极端

- $X, Y$  独立  $\Rightarrow P_{XY} = P_X P_Y \Rightarrow I(X; Y) = 0$ ;
- $X, Y$  相关  $\Rightarrow P_{XY} \neq P_X P_Y \Rightarrow I(X; Y) > 0$ 。

# Mutual Information 是 KL Divergence 的特殊情况

互信息可以写成一个 KL divergence:

$$I(X; Y) = D_{KL}(P_{XY} \parallel P_X P_Y)$$

其中:

- $P_{XY}$  是真实的 **joint distribution**;
- $P_X P_Y$  是假设  $X, Y$  独立时的 joint distribution。

两种极端

- $X, Y$  独立  $\Rightarrow P_{XY} = P_X P_Y \Rightarrow I(X; Y) = 0$ ;
- $X, Y$  相关  $\Rightarrow P_{XY} \neq P_X P_Y \Rightarrow I(X; Y) > 0$ 。

*KL divergence* 是更 *general* 的 “分布差异” 度量;  
*mutual information* 是用 *KL* 衡量 “两个变量离独立有多远” 的特殊形式。

# 四个概念的关系总结

- $H(P)$  —— 真实分布自身的不确定性；最优平均编码长度。
- $H(P, Q)$  —— 数据来自  $P$ ，但按  $Q$  预测/编码的平均损失。
- $D_{KL}(P \parallel Q)$  —— 用  $Q$  代替  $P$  多付出的平均损失。
- $I(X; Y)$  ——  $X$  和  $Y$  共享的信息量；衡量“离独立多远”。

# 四个概念的关系总结

- $H(P)$  —— 真实分布自身的不确定性；最优平均编码长度。
- $H(P, Q)$  —— 数据来自  $P$ ，但按  $Q$  预测/编码的平均损失。
- $D_{KL}(P \parallel Q)$  —— 用  $Q$  代替  $P$  多付出的平均损失。
- $I(X; Y)$  ——  $X$  和  $Y$  共享的信息量；衡量“离独立多远”。

## 两个核心等式

$$H(P, Q) = H(P) + D_{KL}(P \parallel Q)$$

$$I(X; Y) = D_{KL}(P_{XY} \parallel P_X P_Y)$$

# 四个概念的关系总结

- $H(P)$  —— 真实分布自身的不确定性；最优平均编码长度。
- $H(P, Q)$  —— 数据来自  $P$ ，但按  $Q$  预测/编码的平均损失。
- $D_{KL}(P \parallel Q)$  —— 用  $Q$  代替  $P$  多付出的平均损失。
- $I(X; Y)$  ——  $X$  和  $Y$  共享的信息量；衡量“离独立多远”。

## 两个核心等式

$$H(P, Q) = H(P) + D_{KL}(P \parallel Q)$$

$$I(X; Y) = D_{KL}(P_{XY} \parallel P_X P_Y)$$

一句话： *entropy* 是“基本代价”， *cross entropy* 加上“额外代价”就是 *KL*；  
*mutual information* 则是衡量“相关性”的一种 *KL*。

# 谢谢!

有问题吗?